In a scenario when other instances of the command are already in progress, the **`isolatable`** marked command will not execute and will exit with a successful exit code by default. It is possible to specify the exit code the command should exit in such a scenario by specifying the desired status code via the **`isolate`** option in the command.

To specify an exit code of 12 when another instance of the command is already running, specify as follows:

```
--isolated=12
```

Lock Expiry Management

The lock expires in two scenarios, namely:

- The command execution is done, and the lock expires by default.
- The command is interrupted and unable to finish, the lock will expire in 1 hour.

The lock expiration can be modified by defining the **`isolationLockExpiresAt()`** method on the command.

Let's look at the code for the method:

```
use DateTimeInterface;
use DateInterval;

public function isolationLockExpiresAt():
DateTimeInterface|DateInterval
{
  return now()->addMinutes(10);
}
```

The aim of the preceding code is to customize the lock expiration time and to set it to add 10 minutes as the expiration of the lock time.

This enables us to base the expiration of the lock based on the business logic of our application. It is key to note that in case we do not custom specify the time when the lock will expire, the two scenarios are by default incorporated and will take care to release the lock in the given time, as mentioned at the start.

Conclusion

In this chapter, we explored what the Laravel framework is, why it is preferred by developers, and its growing adaptability for application development. We discussed the key pillars of Laravel, namely Scalability, MVC architecture, Community, Eloquent ORM, and Class Dependency management. We looked at